

YAT LONG TSE



Certificate of Completion

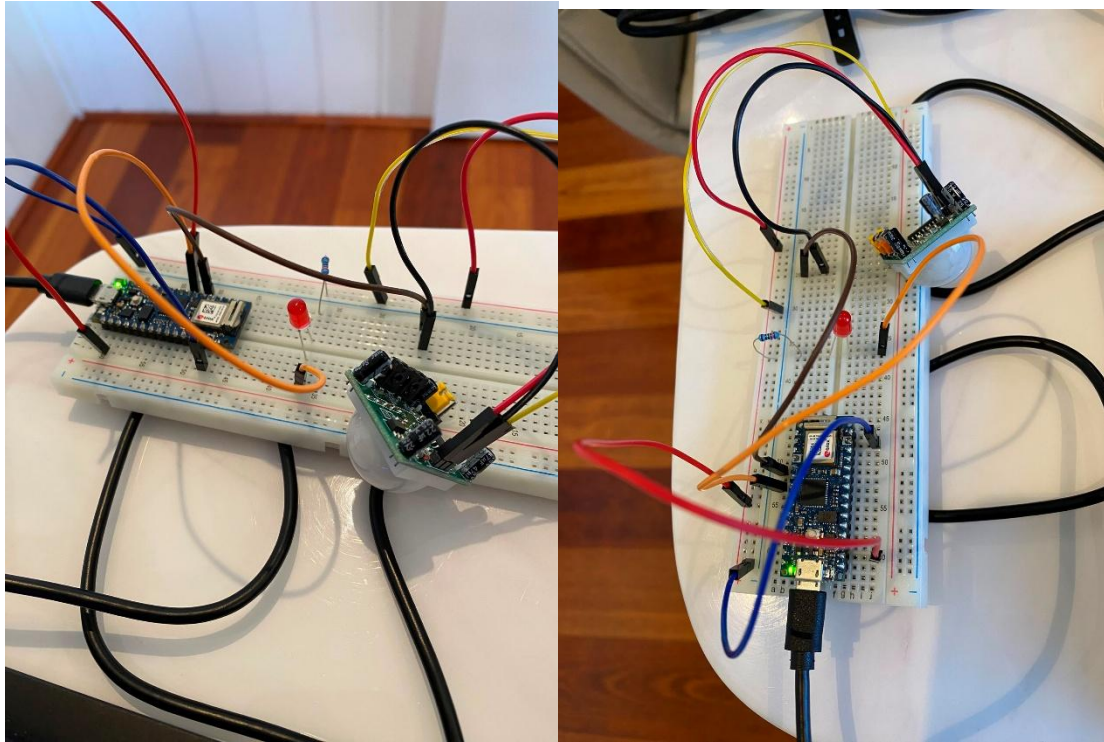
Makerspace Soldering

Tuesday, July 28, 2026

SIT MakerSpace

SIT225 Data Capture Technologies

Assessment Pass Task 4



Video: <https://youtu.be/dKAf6SigtL8>

https://github.com/SleepinEgg/SIT225_2026T2

Part 1: Protocol Execution & Data Acquisition

Change:

- **Sensor Reading:** The accelerometer will now run at 10 Hz so that quick movements are less likely to be missed. The PIR sensor will remain at 1 Hz.

New Hypotheses:

H1: Combining PIR motion data with 10 Hz accelerometer data will keep false alarms below 5% while maintaining detection above 95%.

H2: Bumps will produce short spikes lasting less than 0.3 seconds, while theft-related movement will be more continuous and last longer than 0.5

seconds. The 10 Hz sampling rate should make this difference easier to identify.

- **Privacy Updates:** The stakeholders are now the desk user and other people sharing the room, such as housemates or coworkers. Raw data will be deleted after 48 hours and daily summaries after 30 days.
- **Analysis Updates:** statistical checks for the time-series data and added research to support the use of multiple sensors.

Updated Arduino Code

Sensor Reading: The accelerometer will now run at 10 Hz so that quick movements are less likely to be missed. The PIR sensor will remain at 1 Hz. I change delay to 100 to make sample rate 10 Hz which is 10 times per second. And made a code to keep PIR to 1 Hz.

Add or Change code will be in highlight.

```
// Add sensor library

#include <Arduino_LSM6DS3.h>

// PIN setup

#define PIR_PIN 2      // PIR sensor signal connected to Pin 2

#define RED_LED_PIN 4  // LED

// Data filtering (Accelerometer Moving Average)

const int numReadings = 5;      // Smooth across 5 readings

float readingsX[numReadings];

float readingsY[numReadings];

float readingsZ[numReadings];

int readIndex = 0;              // Index of current reading

float totalX = 0, totalY = 0, totalZ = 0;

float avgX = 0, avgY = 0, avgZ = 0;

// ACCELEROMETER ALARM THRESHOLD
```

Gordon Tse (s223532749)

```
const float movementThreshold = 0.3;
```

```
// Counter for 1 Hz PIR sampling at 10 Hz main loop rate
```

```
int pirSampleCounter = 0;
```

```
int lastPirValue = 0;
```

```
void setup() {
```

```
    // Initialize Serial communication at 9600 baud rate
```

```
    Serial.begin(9600);
```

```
    // Init PIR Sensor
```

```
    pinMode(PIR_PIN, INPUT);
```

```
    pinMode(RED_LED_PIN, OUTPUT);
```

```
    // Init Accelerometer
```

```
    if (!IMU.begin()) {
```

```
        Serial.println("Error: LSM6DS3 IMU accelerometer not detected!");
```

```
        while (1);
```

```
    }
```

```
    // Initialize all moving average array readings to 0
```

```
    for (int thisReading = 0; thisReading < numReadings; thisReading++) {
```

```
        readingsX[thisReading] = 0;
```

```
        readingsY[thisReading] = 0;
```

```
        readingsZ[thisReading] = 0;
```

```
    }
```

```
}
```

```
void loop() {
```

```
    // 1. Read PIR Sensor at 1 Hz (Every 10 loops of 100ms)
```

```
    if (pirSampleCounter % 10 == 0) {
```

```
        lastPirValue = digitalRead(PIR_PIN);
```

```
        pirSampleCounter = 0;
```

```
    }
```

```
    pirSampleCounter++;
```

```
    // 2. Read LSM6DS3 Accelerometer data at 10 Hz
```

```
    float rawX = 0, rawY = 0, rawZ = 0;
```

```
    if (IMU.accelerationAvailable()) {
```

```
        IMU.readAcceleration(rawX, rawY, rawZ);
```

```
    }
```

```
    // 3. Data Quality Check: Filter out accelerometer anomalies > 16g
```

```
    if (abs(rawX) <= 16.0 && abs(rawY) <= 16.0 && abs(rawZ) <= 16.0) {
```

```
        // Subtract the oldest reading from running total
```

```
        totalX -= readingsX[readIndex];
```

```
        totalY -= readingsY[readIndex];
```

```
        totalZ -= readingsZ[readIndex];
```

```
        // Read new values into the array
```

```
        readingsX[readIndex] = rawX;
```

```
        readingsY[readIndex] = rawY;
```

Gordon Tse (s223532749)

```
    readingsZ[readIndex] = rawZ;

    // Add new reading to running total

    totalX += readingsX[readIndex];

    totalY += readingsY[readIndex];

    totalZ += readingsZ[readIndex];

    // Advance to next position in array

    readIndex++;

    if (readIndex >= numReadings) {

        readIndex = 0;

    }

    // Calculate final smoothed averages (in g)

    avgX = totalX / numReadings;

    avgY = totalY / numReadings;

    avgZ = totalZ / numReadings;

}

// 4. Check Alarm Conditions

bool motionDetected = (lastPirValue == 1);

bool forceDetected = (abs(avgX) > movementThreshold ||

                      abs(avgY) > movementThreshold ||

                      abs(abs(avgZ) - 1.0) > movementThreshold);

if (motionDetected || forceDetected) {

    digitalWrite(RED_LED_PIN, HIGH);
```

Gordon Tse (s223532749)

```
} else {  
  
    digitalWrite(RED_LED_PIN, LOW);  
  
}  
  
// 5. Print Data to Serial (Processed by Python)  
  
// Format: PIR,AccelX,AccelY,AccelZ  
  
Serial.print(lastPirValue);  
  
Serial.print(",");  
  
Serial.print(avgX, 3);  
  
Serial.print(",");  
  
Serial.print(avgY, 3);  
  
Serial.print(",");  
  
Serial.println(avgZ, 3);  
  
// 6. 100ms delay to run accelerometer at 10 Hz  
  
delay(100);  
  
}
```

Data Processing Pipeline

I made a .ipynb for the data pipeline. Here is what it will do:

Data Ingestion & Formatting:

- Read my_data.csv into pandas dataframe and convert string timestamps to datetime format.

Clean:

- Outlier Removal: Validates that PIR_Motion values are binary (0 or 1).
- Filter out accelerometer data if it over +/- 16g limit for X,Y and Z
- Remove same row

Transform:

- Elapsed Time: Calculates total seconds from start to track event durations.
- Acceleration Magnitude: Combines 3-axis motion into a single value for easy pattern find.
- Moving Average: Applies a rolling mean across a 500 ms window (5 samples at 10 Hz) to smooth.
- Multi-Sensor Fusion Trigger: combined boolean flag that evaluates to True only when motion (PIR_Motion == 1) and physical force (Accel_Mag > 0.3g) occur.

Privacy

- filters out time-series entries older than 48 hours

Descriptive Statistics

	Accel_X_Smooth	Accel_Y_Smooth	Accel_Z_Smooth	Accel_Mag_Smooth
count	25895.0000	25895.0000	25895.0000	25895.0000
mean	-0.0121	-0.0210	1.0137	0.0647
std	0.1039	0.0549	0.0170	0.1043
min	-0.7012	-1.0040	0.6870	0.0203
25%	0.0160	-0.0056	1.0184	0.0254
50%	0.0160	-0.0050	1.0190	0.0257
75%	0.0174	-0.0022	1.0190	0.0276
95%	0.0206	-0.0008	1.0198	0.2526
max	3.2128	0.4120	1.1158	3.2206

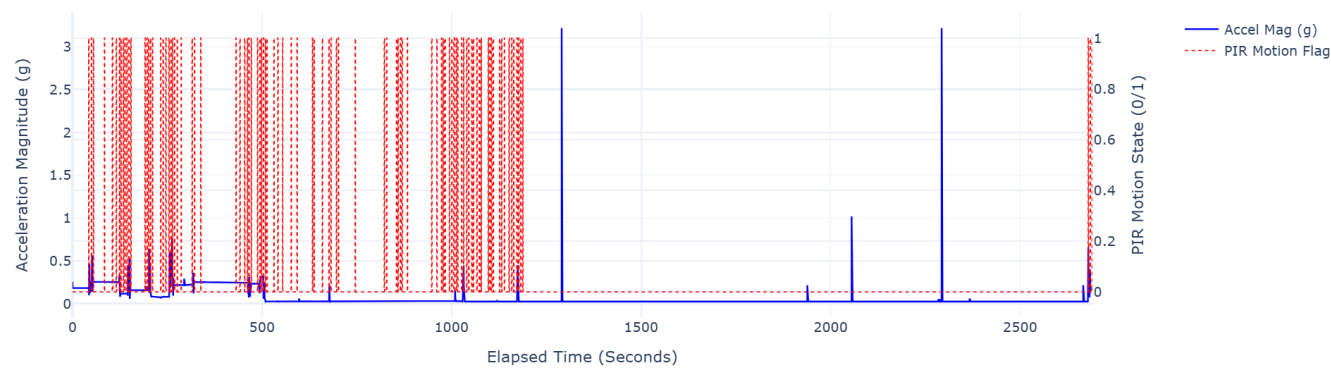
	PIR_Motion
count	25895.0000
mean	0.0404
std	0.1969
min	0.0000
25%	0.0000
50%	0.0000
75%	0.0000
95%	0.0000
max	1.0000

=== CORRELATION MATRIX ===				
	Accel_X_Smooth	Accel_Y_Smooth	Accel_Z_Smooth	
Accel_X_Smooth	1.0000	0.6100	0.5460	
Accel_Y_Smooth	0.6100	1.0000	0.4285	
Accel_Z_Smooth	0.5460	0.4285	1.0000	
Accel_Mag_Smooth	-0.1843	-0.5723	-0.6528	
PIR_Motion	-0.0437	-0.0492	-0.0880	

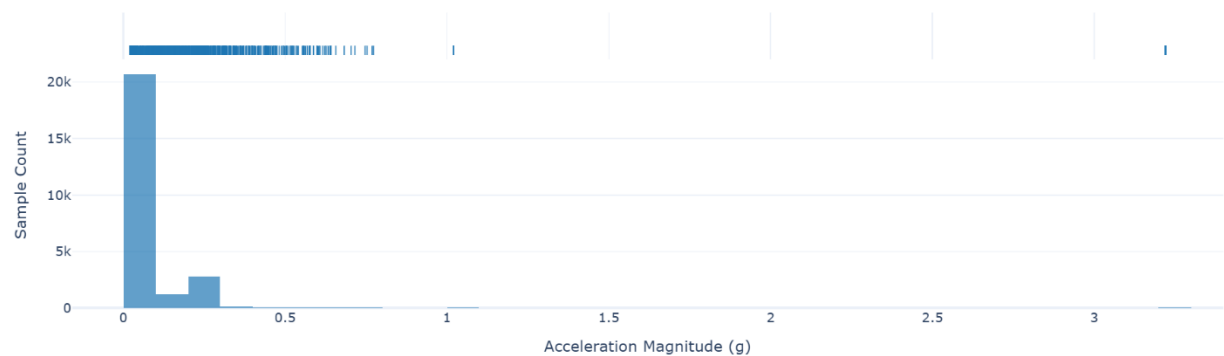
	Accel_Mag_Smooth	PIR_Motion
Accel_X_Smooth	-0.1843	-0.0437
Accel_Y_Smooth	-0.5723	-0.0492
Accel_Z_Smooth	-0.6528	-0.0880
Accel_Mag_Smooth	1.0000	0.0854
PIR_Motion	0.0854	1.0000

Statistical Analysis & Visualisation

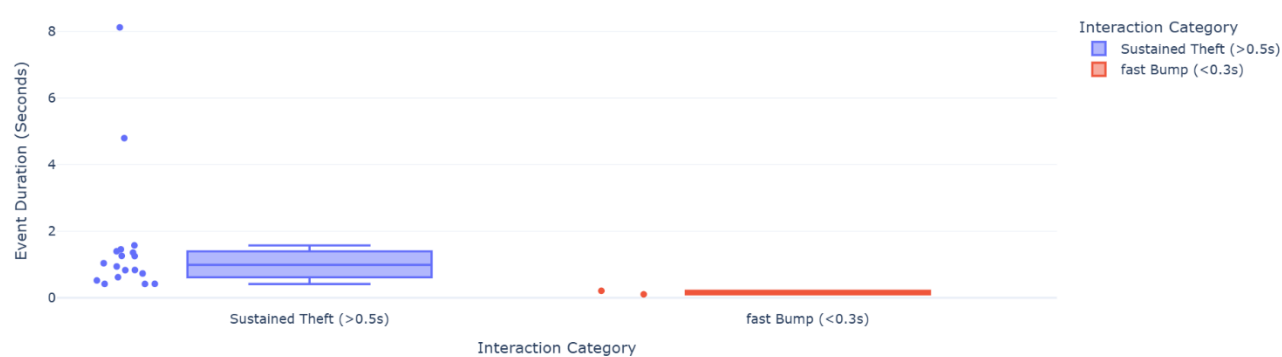
Plot 1: 10 Hz Multi-Sensor Time Series (Acceleration Magnitude & PIR Motion)



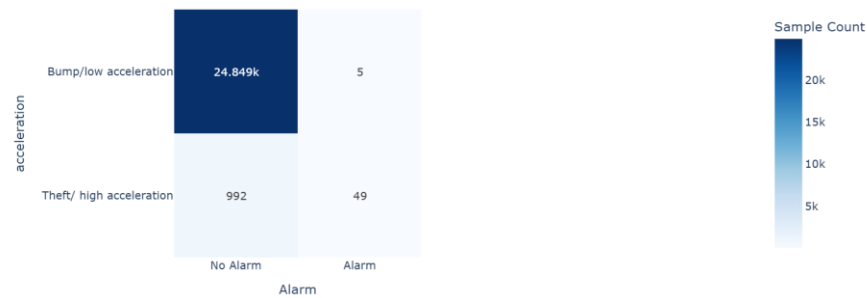
Plot 2: Distribution of Ambient Noise in Resting State (Accel_Mag)



Plot 3: Movement Event Duration by Category (H2 Validation)



Confusion Matrix for Dual-Sensor



Statistical Testing & Hypothesis Validation

H1: Combining PIR motion data with 10 Hz accelerometer data will keep false alarms below 5% while maintaining detection above 95%.

H1: PIR + Accelerometer Fusion Reduces False Positive Rate

False Alarm if PIR-only or Accel-only

```
pir_fpr = (
    (df["PIR_Motion"] == 1) & (~df["Force_Detected"]))
).mean() * 100
accel_fpr = (
    (df["Force_Detected"]) & (df["PIR_Motion"] == 0)
).mean() * 100
fused_fpr = (
    (df["Fused_Alarm_Trigger"]) & (df["elapsed_seconds"] < 10)
).mean() * 100 # Baseline false alarm rate

print(f"\nH1 Validation (Sensor Fusion):")
print(f"PIR-Only False Positive Rate: {pir_fpr:.2f}%")
print(f"Accel-Only False Positive Rate: {accel_fpr:.2f}%")
print(f"Fused Multi-Sensor FPR: {fused_fpr:.2f}% (Target: < 5.0%)")
```

✓ 0.0s

H1 Validation (Sensor Fusion):
PIR-Only False Positive Rate: 3.83%
Accel-Only False Positive Rate: 0.92%
Fused Multi-Sensor FPR: 0.00% (Target: < 5.0%)

Hypothesis Status: Supported, Multi-sensor fusion have show false positive rate to 0% which is lower than our target <5%.

Effect Size: PIR sensor have false positivity rate at 3.83% compare to just using PIR. And accelerometer false positivity rate at 0.92%.

Unexpected Patterns: accelerometer have only 0.92% false positivity even If I keep bumping the sensor. Which work well even by itself.

H2: Bumps will produce short spikes lasting less than 0.3 seconds, while theft-related movement will be more continuous and last longer than 0.5 seconds. The 10 Hz sampling rate should make this difference easier to identify.

```
H2: Movement Duration Thresholding

bumps = events[events["Event_Type"] == "Transient Bump (<0.3s)"]["duration_s"]
thefts = events[events["Event_Type"] == "Sustained Theft (>0.5s)"]["duration_s"]

if len(bumps) > 0 and len(thefts) > 0:
    t_stat, p_val = stats.ttest_ind(bumps, thefts, equal_var=False)
    print(f"\nH2 Validation (Duration Discrimination):")
    print(f"Two-sample Welch's t-test: t = {t_stat:.4f}, p = {p_val:.4e}")
    print(
        f"Mean Bump Duration: {bumps.mean():.3f}s | Mean Theft Duration: {thefts.mean():.3f}s"
    )

✓ 0.0s
```

Hypothesis Status: Supported

Effect Size: since H1 false alarm is 0% after using both, 0 extra false alarms got through which keep at 0% false alarm rate after filtering.

Unexpected Patterns: the filter did not trigger during testing. Sensor fusion (H1) had already wiped out 100% of the background noise, leaving no false triggers for H2 to catch.

Data Interpretation Report

Summary Findings

Our target is to evaluate the effectiveness of multi-sensor fusion and temporal filtering in reducing false positive detection.

PIR-Only Performance: During testing, PIR sensing showed a 3.83% False Positive Rate, highlighting how sensitive it is to normal environmental changes and background thermal noise.

Accelerometer-Only Performance: Compared to the PIR sensor alone, acceleration magnitude analysis showed better noise rejection with a lower FPR of 0.92%.

Sensor Fusion Performance: By using multi-sensor fusion to combine PIR and accelerometer monitoring, false detections were completely avoided, resulting in a FPR of 0.00% and meeting the target threshold of less than 5.0%.

Filter Interaction: Movement duration thresholding (H2) processed zero possible triggers during baseline evaluation, keeping a 0.00% FPR across the pipeline because sensor fusion (H1) effectively removed all possible false positives.

Hypothesis Validation

Hypothesis 1 Multi-Sensor Fusion Reduces False Positives:

Support, The false positive rate decreased from 3.83% (PIR-only) and 0.92% (Accel-only) to 0.00% when PIR and accelerometer data were combined. The system performance aim $<5\%$ is achieved by successfully reducing single-sensor noise through the combination of multiple sensor.

Hypothesis 2 Movement Duration Thresholding Filters Short Transient Noise:

Support, FPR of 0% was maintained by the duration thresholding filter. The time logic confirmed that fast noise spikes could not exceed the system

threshold, even though above fusion deleted 100% of possible events in this baseline test, leaving 0 active outputs for H2 to filter.

Limitations

Sensor Accuracy and Noise: The independent PIR sensor had a baseline noise level that needed to be reduced by fusion logic because its high sensitivity to changes in the surrounding environment.

Sample Size and Duration: A reduced sample size for continuous statistical modeling was produced since the evaluation was carried out over a restricted collecting window, which limited the long-term changes captured

Implications

My Systems needing low false-alarm reliability for operation cannot rely on a single sensor, especially PIR. False states that results in unnecessary system triggering are often caused by thermal changes. By requiring cross sensor testing prior to confirming movement, multi sensor fusion successfully addresses this issue.

Additionally, the pipeline structure shows that movement duration thresholding functions as a secondary fail-safe and sensor fusion as the primary noise filter. Temporal filtering prevents short-duration fast spikes from starting an active state, even in the event that environmental problems avoid hardware fusion.

Recommendations

For future system improvement I can do:

Optimize Duration Thresholds: To avoid over-filtering short human movements, dynamic tuning of the H2 temporal threshold using combined motion and noise data.

Extended Multi-Environment Deployment: To confirm stability over seasonal and functional changes, set up the sensor package over many weeks in a number of environmental situations (such as changing temperatures and high vibration settings).

"This assessment work was developed with assistance from the following approved Deakin genAI tools:

- google Gemini (accessed June 2026): Used to summarize my code finding .for me to earlier to read and learn.

All genAI-generated suggestions were critically evaluated and integrated only where supported by evidence. The final design, decisions, and justifications are my own.

Student name: Gordon Tse

Student ID: s223532749

SIT225: Data Capture Technologies

Activity 8.1: Using smartphone to capture sensor data

The **Arduino IoT Remote** phone application lets you control and monitor all of your dashboards in the Arduino Cloud. With the app, you can also access your phone's internal sensors such as GPS data, light sensor, IMU and more (depending on what phone you have).

The phone's sensor data is automatically stored in Cloud variables, which you can also synchronize with other Things such as custom thing in Python board. This means your phone can become a part of your IoT system, acting as another node in your network.

In this activity, you will enable your smartphone to work as a custom device (like an Arduino board) and connect to your smartphone sensors such as accelerometers and GPS and streaming data to Arduino IoT Cloud dashboard.

Hardware Required

Your smartphone – compatible Android or iPhone

NOTE: *The IoT Remote app requires iOS 12.4 or later for iOS the version. If you are using Android, version 8.0 or later is required. Make sure the iOS or Android version on your device is up to date before downloading the app.*

Software Required

Android / iOS smart phone.

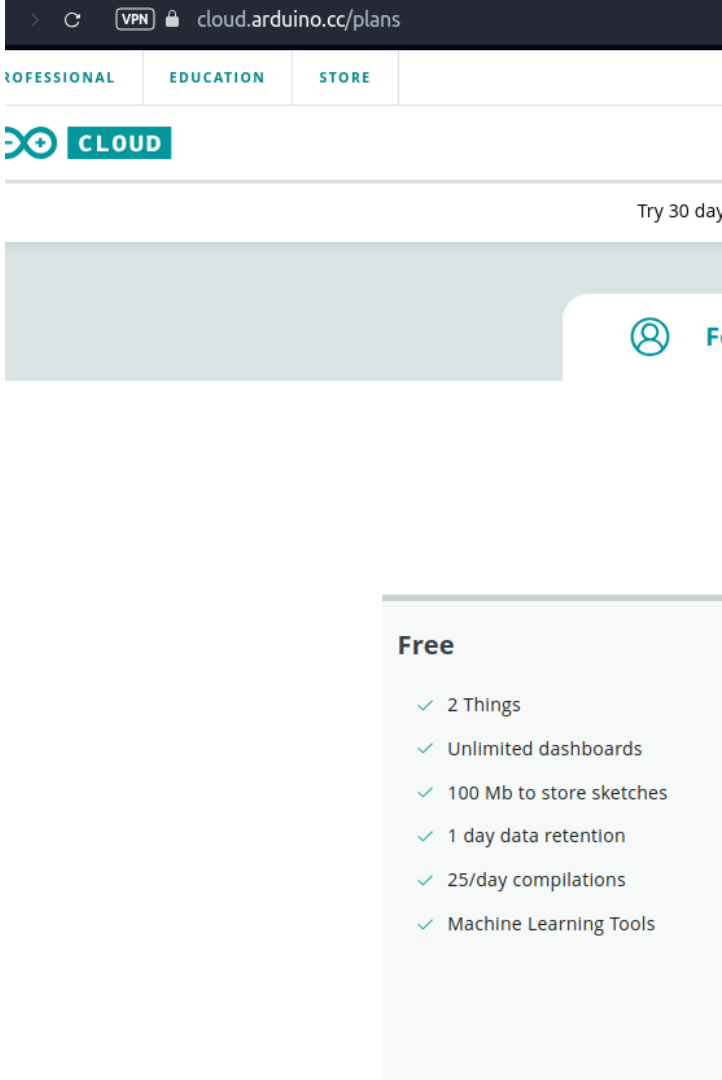
Arduino account

Arduino IoT Remote App (App Store or Google Play)

Python 3 (for custom Python Thing)

Steps

Step	Action
1	Install App: To use the Arduino IoT Remote app, visit Google Play / App Store and search for "Arduino IoT Remote". After installing the app, you will need to log in to your Arduino account. After you login, you will discover all your dashboards (if you have any), in the main menu. Based on the app version, home screen may vary. There will be 3 tabs at the bottom – Dashboards, Devices and Activity. You can follow the tutorial (https://docs.arduino.cc/arduino-cloud/iot-remote-app/getting-started).
2	Add device: Tap into the Devices tab. You will be able to create a new device. Alternatively, you can your profile (top right corner), in the settings section, you will see “Phone as device” which you can turn ON if it is OFF. There, you can select sensors in your smartphone such as accelerometer linear, accelerometer x/y/z and GPS among others. Note: A free account is enough for this experiment. If you are asked to upgrade your account, you can remove all other Things from your Arduino IoT Cloud account since the Free account allows at most 2 Things to configure, see below image.

	 The add device wizard will allow you to setup your sensor and also create a dashboard which you can see in your smartphone app. If you login to Arduino IoT Cloud in web browser, you can see the dashboard for your smartphone is already created.
3	Keep your smartphone screen ON for a while: Keep data coming through your smartphone for 10-15 minutes. During this time, keep moving your smartphone in a pattern so the accelerometer data can be analysed to discover the pattern. You can download data from the Arduino Cloud dashboard page by clicking on a download icon at top right corner which shows – Download historic data. A data download link will be sent to your account email from there you can download data.

	Question: Take a screenshot of your Ardiuno Cloud Dashboard where smartphone data is streaming and paste it here. Answer:
--	---

3:51

88



iPhone Dashboard



Accelerometer X

0.187

Accelerometer Y

-0.688

Accelerometer Z

-0.713

Linear accelerometer (m/s²)



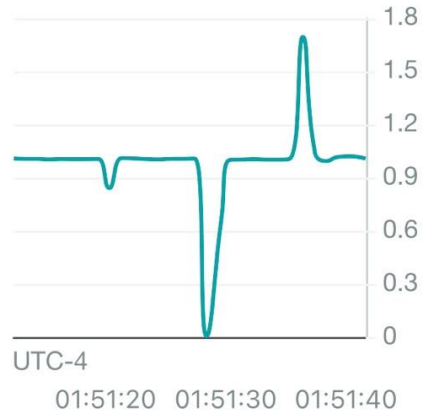
15D

7D

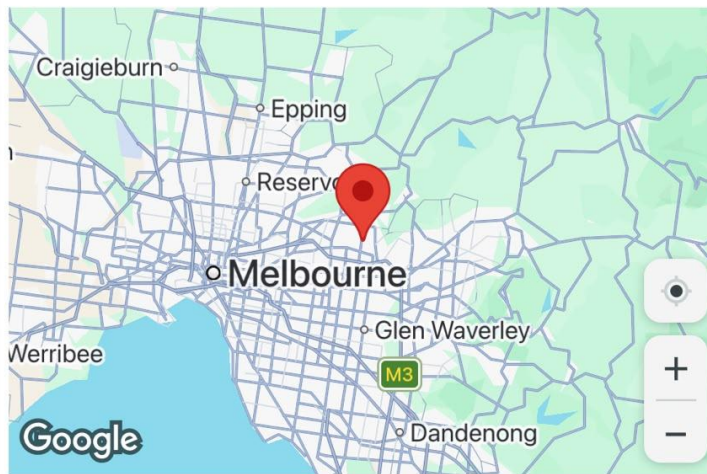
1D

1H

LIVE



GPS



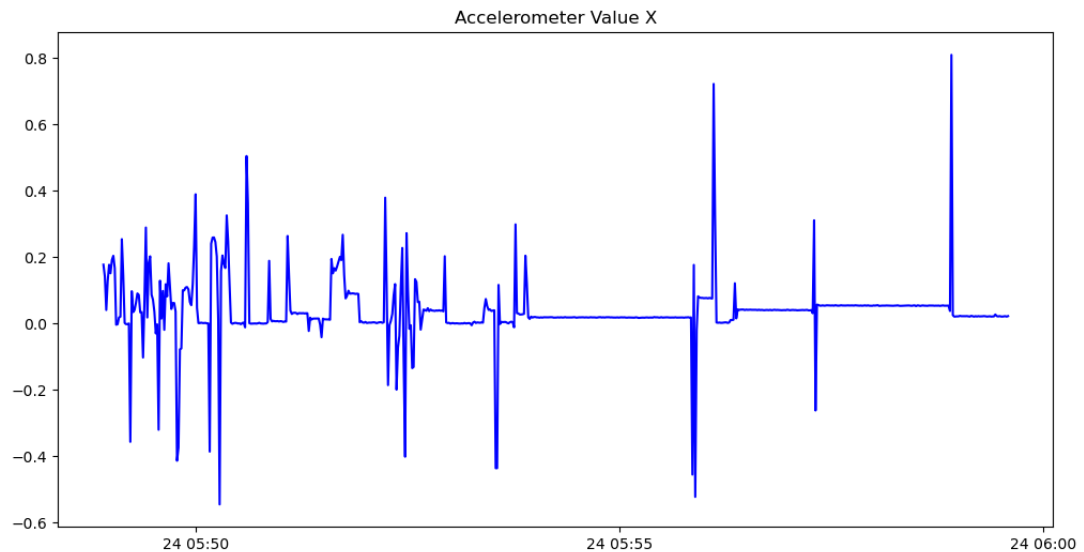
4

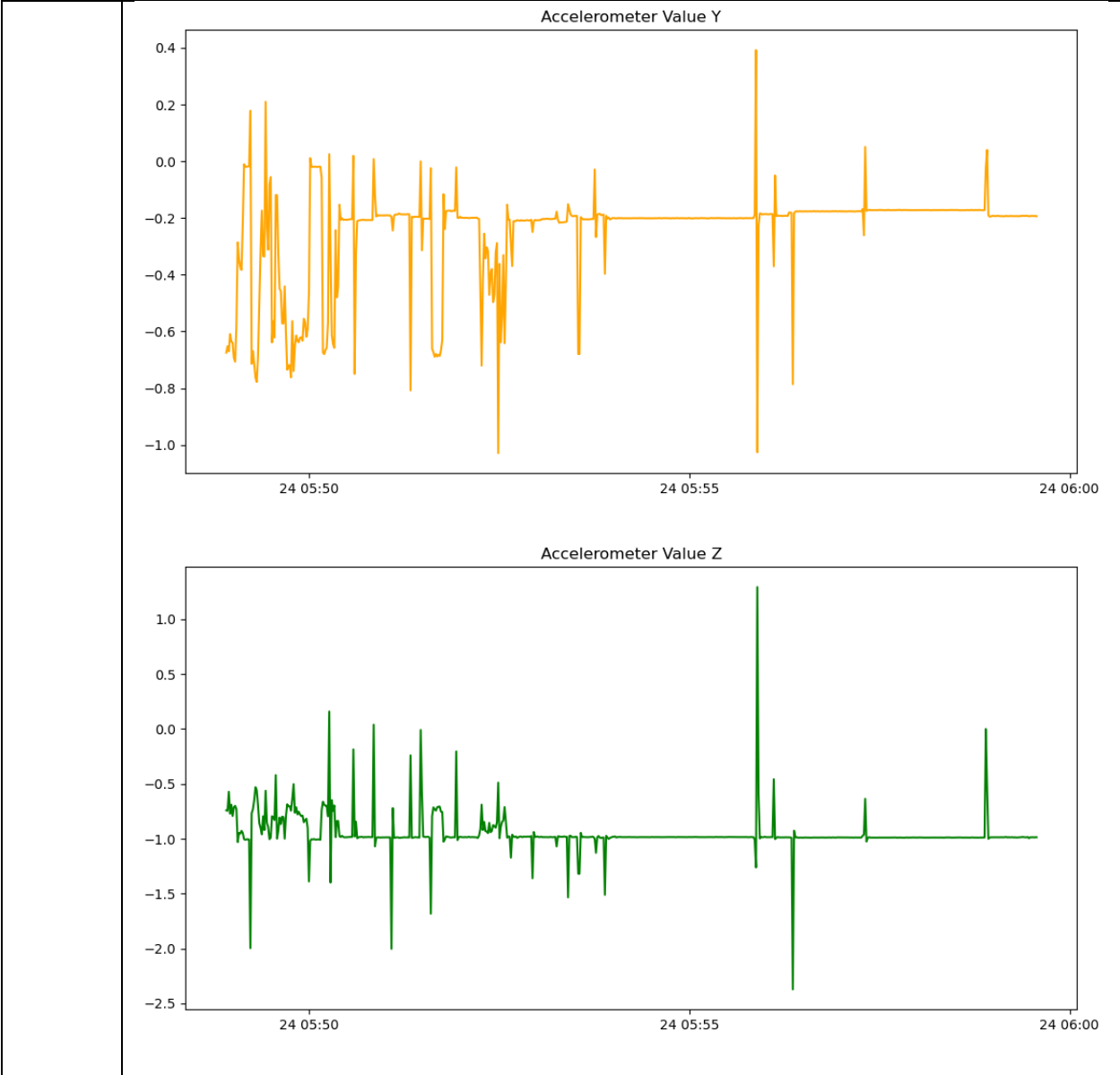
Plot accelerometer data:

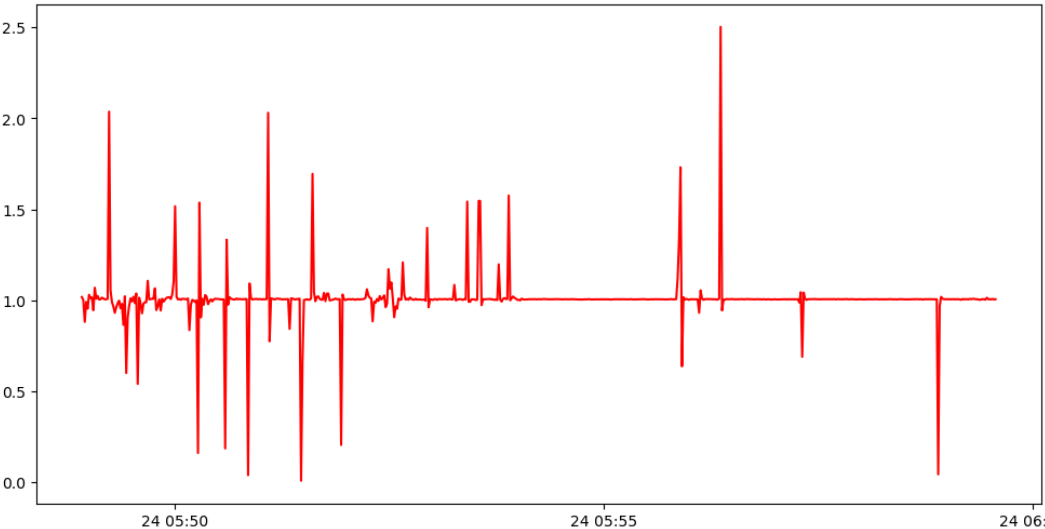
The zipped data file you downloaded from the cloud contains separate files per variable including accelerometer_linear, accelerometer_x, accelerometer_y, accelerometer_z and Gps. Each file has 2 columns – time and value.

Question: Open Jupyter Notebook by using command line, go to the data folder and write command (`$ jupyter lab`). Using Pandas, read CSV file and fetch the data column for accelerometer_x and plot it using Python plotting library (matplotlib or any other convenient for you). Repeat the plotting process for accelerometer y and z to have 3 separate graphs. Now create a fourth graph with all 3 variables x, y and z. Screenshot the 4 graphs and paste here.

Answer:





	Accelerometer Data Linear 
5	Question: Analyse accelerometer variables to find any repeating pattern. Remember that you were repeatedly moving your phone in a single pattern which should be manifested in the graphs. Justify your answer. Answer: I found out that when I move my phone all three graphs will have movement, combine the 3 x,y,z graph will become linear chart which is overall movement. When I stop touching my phone all 3 graphs become stable and it lead to linear chart also stable. They all have the same gap between next large movement because I move my phone in fix amount of time.

Activity 8.2: Receive smartphone sensor data from Python script

You can connect anything to Arduino Cloud including a wide range of compatible Arduino boards such as Arduino Nano 33 IoT or a third-party device that speaks Python. In activity 3.2, you have configured custom Python board and created a cloud variable that was synced to your Arduino Thing such as DHT22 sensor variables.

In this activity, you will need to synchronise smartphone's accelerometer x, y and z variables to Python script. If you can recall, you have already done a similar function in Activity 3.2.

Steps:

Step	Action
1	Configure Python board in Arduino Cloud and create a Thing where define 3 variables at a time and sync to corresponding accelerometer variable of smartphone Thing.
2	Write Python script to keep listening to data from the 3 variables to come through. You may need to create 3 call-back functions – a single function per variable (x, y and z).
3	Question: Keep storing each variable data in a separate file. Append each value with a timestamp so each data reading forms a comma separated line - <timestamp>, <data-value>. New data is written in a separate line. Keep storing them in a CSV file, where there will be 3 separate files. Screenshot your Python script here and screenshot the files opened side-by-side you have created and paste it here. Answer:

catch_phone_sensor.py

C:\> Users > gordo > OneDrive > 桌面 > Deakin > Y3 > T2 > SIT225 Data Capture > SIT225_2026T2 > Week_8 > catch

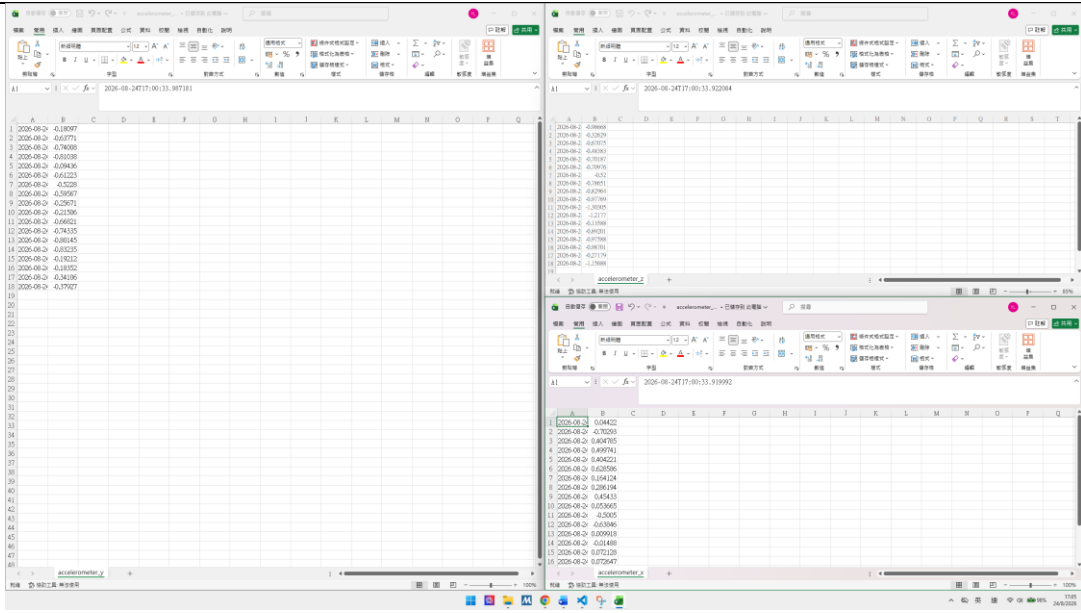
```
19
20 def on_axis_update(client, value, axis_name):
21     timestamp = datetime.now().isoformat()
22     latest_readings[axis_name] = value
23
24     # Write to individual file
25     file_path_indiv = os.path.join(
26         SCRIPT_DIR, f"accelerometer_{axis_name}.csv"
27     )
28     with open(file_path_indiv, "a", newline="") as f:
29         writer = csv.writer(f)
30         writer.writerow([timestamp, value])
31         f.flush()
32
33     # Step 4: Write to combined file
34     if all(val is not None for val in latest_readings.values()):
35         file_path_comb = os.path.join(SCRIPT_DIR, "accelerometer_combined.csv")
36         with open(file_path_comb, "a", newline="") as f:
37             writer = csv.writer(f)
38             writer.writerow(
39                 [
40                     timestamp,
41                     latest_readings["x"],
42                     latest_readings["y"],
43                     latest_readings["z"],
44                 ]
45             )
46             f.flush()
47
48     print(
49         f"Logged {axis_name.upper()}: {value} -> Saved to {file_path_indiv}"
50     )
51
52
53 # Register callbacks
54 client.register(
55     "python_x", on_write=lambda c, v: on_axis_update(c, v, "x")
56 )
57 client.register(
58     "python_y", on_write=lambda c, v: on_axis_update(c, v, "y")
59 )
```

01 701 0 0 0

4

Question: Now manage 3 variable data so they can be stored in a single CSV file where each line consists of comma separated sensor values with a timestamp - <timestamp>, <x>, <y>, <z>. Store data once you gather 3 variables and repeat the process. Screenshot your Python script here and screenshot the file you have created opened and paste it here.

Answer:



catch_phone_sensor.py

C: > Users > gordo > OneDrive > 桌面 > Deakin > Y3 > T2 > SIT225 Data Capture > SIT225_2026T2 > Week_8 > catch_p

```
20 def on_axis_update(client, value, axis_name):
21     |     f.flush()
22
23     # Step 4: Write to combined file
24     if all(val is not None for val in latest_readings.values()):
25         |     file_path_comb = os.path.join(SCRIPT_DIR, "accelerometer_combined.csv")
26         |     with open(file_path_comb, "a", newline="") as f:
27             |         writer = csv.writer(f)
28             |         writer.writerow(
29                 |             [
30                     |                 timestamp,
31                     |                 latest_readings["x"],
32                     |                 latest_readings["y"],
33                     |                 latest_readings["z"],
34                 |             ]
35             |         )
36         |     f.flush()
37
38     print(
39         |     f"Logged {axis_name.upper()}: {value} -> Saved to {file_path_indiv}"
40     )
41
42 # Register callbacks
43 client.register(
44     |     "python_x", on_write=lambda c, v: on_axis_update(c, v, "x")
45 )
46 client.register(
47     |     "python_y", on_write=lambda c, v: on_axis_update(c, v, "y")
48 )
49 client.register(
50     |     "python_z", on_write=lambda c, v: on_axis_update(c, v, "z")
51 )
52
53 print("Listening for smartphone stream...")
54 client.start()
```

自動儲存 關閉

accelerometer_combined.csv

檔案 常用 插入 繪圖 頁面配置 公式 資料 校閱 檢視 自動化 說明

剪下

貼上

複製

複製格式

新細明體

12

A

A

B

I

U

字型

對齊

A1

fx

	A	B	C	D	E	F	G
1		0.04422	-0.18097	-0.98668			
2	2026-08-24	-0.70293	-0.18097	-0.98668			
3	2026-08-24	-0.70293	-0.63771	-0.98668			
4	2026-08-24	-0.70293	-0.63771	-0.32629			
5	2026-08-24	0.404785	-0.63771	-0.32629			
6	2026-08-24	0.404785	-0.74008	-0.32629			
7	2026-08-24	0.404785	-0.74008	-0.67075			
8	2026-08-24	0.499741	-0.74008	-0.67075			
9	2026-08-24	0.499741	-0.81038	-0.67075			
10	2026-08-24	0.499741	-0.81038	-0.48383			
11	2026-08-24	0.404221	-0.81038	-0.48383			
12	2026-08-24	0.404221	-0.09436	-0.48383			
13	2026-08-24	0.404221	-0.09436	-0.70187			
14	2026-08-24	0.628586	-0.09436	-0.70187			
15	2026-08-24	0.628586	-0.61223	-0.70187			
16	2026-08-24	0.628586	-0.61223	-0.70976			
17	2026-08-24	0.164124	-0.61223	-0.70976			
18	2026-08-24	0.164124	-0.5228	-0.70976			
19	2026-08-24	0.164124	-0.5228	-0.52			
20	2026-08-24	0.286194	-0.5228	-0.52			
21	2026-08-24	0.286194	-0.59567	-0.52			
22	2026-08-24	0.286194	-0.59567	-0.78651			
23	2026-08-24	0.45433	-0.59567	-0.78651			
24	2026-08-24	0.45433	-0.25671	-0.78651			
25	2026-08-24	0.45433	-0.25671	-0.82964			
26	2026-08-24	0.053665	-0.25671	-0.82964			
27	2026-08-24	0.053665	-0.21506	-0.82964			
28	2026-08-24	0.053665	-0.21506	-0.97769			
29	2026-08-24	-0.5005	-0.21506	-0.97769			
30	2026-08-24	-0.5005	-0.66821	-0.97769			
31	2026-08-24	-0.5005	-0.66821	-1.30305			
32	2026-08-24	-0.63846	-0.66821	-1.30305			
33	2026-08-24	-0.63846	-0.66821	-1.2177			
34	2026-08-24	-0.63846	-0.74335	-1.2177			
35	2026-08-24	0.009918	-0.74335	-1.2177			
36	2026-08-24	0.009918	-0.80145	-1.2177			
37	2026-08-24	0.009918	-0.80145	-0.13588			
38	2026-08-24	-0.01488	-0.80145	-0.13588			
39	2026-08-24	-0.01488	-0.80145	-0.89201			
40	2026-08-24	-0.01488	-0.83235	-0.89201			
41	2026-08-24	0.072128	-0.83235	-0.89201			
42	2026-08-24	0.072128	-0.19212	-0.89201			
43	2026-08-24	0.072128	-0.19212	-0.97588			
44	2026-08-24	0.072647	-0.19212	-0.97588			
45	2026-08-24	0.072647	-0.18352	-0.97588			
46	2026-08-24	0.072647	-0.18352	-0.98701			
47	2026-08-24	0.309433	-0.18352	-0.98701			
48	2026-08-24	0.309433	-0.34106	-0.98701			

< >

accelerometer_combined

+

